

**AIM:**

To implement the simple substitution technique named Caesar cipher using C language.

**ALGORITHM:**

STEP-1: Read the plain text from the user.

STEP-2: Read the key value from the user.

STEP-3: If the key is positive then encrypt the text by adding the key with each character in the plain text.

STEP-4: Else subtract the key from the plain text.

STEP-5: Display the cipher text obtained above.

**PROGRAM:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
int main()
```

```
{
```

```
    char plain[100], cipher[100];
```

```
    int key, i, length;
```

```
    printf("Enter the plain text: ");
```

```
    scanf("%s", plain);
```

```
    printf("Enter the key value: ");
```

```
    scanf("%d", &key);
```

```
    key = key % 26;
```

```
length = strlen(plain);
```

```
printf("\nPLAIN TEXT    : %s", plain);
```

```
printf("\nENCRYPTED TEXT : ");
```

```
for(i = 0; i < length; i++)
```

```
{
```

```
    if(isupper(plain[i]))
```

```
    {
```

```
        cipher[i] = ((plain[i] - 'A' + key) % 26) + 'A';
```

```
    }
```

```
    else if(islower(plain[i]))
```

```
    {
```

```
        cipher[i] = ((plain[i] - 'a' + key) % 26) + 'a';
```

```
    }
```

```
    else
```

```
    {
```

```
        cipher[i] = plain[i];
```

```
    }
```

```
    printf("%c", cipher[i]);
```

```
}
```

```
cipher[length] = '\0';
```

```
printf("\nDECRYPTED TEXT : ");
```

```
for(i = 0; i < length; i++)
```

```
{
    if(isupper(cipher[i]))
    {
        plain[i] = ((cipher[i] - 'A' - key + 26) % 26) + 'A';
    }
    else if(islower(cipher[i]))
    {
        plain[i] = ((cipher[i] - 'a' - key + 26) % 26) + 'a';
    }
    else
    {
        plain[i] = cipher[i];
    }

    printf("%c", plain[i]);
}

plain[length] = '\0';

return 0;}
```

## **OUTPUT:**

```
Enter the plain text: KESHAVARAM
Enter the key value: 6

PLAIN TEXT      : KESHAVARAM
ENCRYPTED TEXT   : QKYNGBGXGS
DECRYPTED TEXT   : KESHAVARAM

...Program finished with exit code 0
Press ENTER to exit console.[]
```

## **RESULT:**

Thus the implementation of Caesar cipher had been executed successfully.

## **EXP.N0:2**

## **IMPLEMENTATION OF PLAYFAIR CIPHER**

### **AIM:**

To write a C program to implement the Playfair Substitution technique.

### **ALGORITHM:**

STEP-1: Read the plain text from the user.

STEP-2: Read the keyword from the user.

STEP-3: Arrange the keyword without duplicates in a 5\*5 matrix in the row order and fill the remaining cells with missed out letters in alphabetical order. Note that „i“ and „j“ takes the same cell.

STEP-4: Group the plain text in pairs and match the corresponding corner letters by forming a rectangular grid.

STEP-5: Display the obtained cipher text.

### **PROGRAM:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define SIZE 5
```

```
char keyMatrix[SIZE][SIZE];
```

```
void generateKeyMatrix(char key[])
```

```
{
```

```
    int used[26] = {0};
```

```
int i, j = 0, k = 0;
```

```
used['J' - 'A'] = 1;
```

```
for (i = 0; key[i] != '\0'; i++)
```

```
{
```

```
    char ch = toupper(key[i]);
```

```
    if (ch == 'J')
```

```
        ch = 'I';
```

```
    if (!used[ch - 'A'])
```

```
    {
```

```
        keyMatrix[j][k++] = ch;
```

```
        used[ch - 'A'] = 1;
```

```
    if (k == SIZE)
```

```
    {
```

```
        k = 0;
```

```
        j++;
```

```
    }
```

```
}
```

```
}
```

```
for (i = 0; i < 26; i++)
```

```
{  
    if (!used[i])  
    {  
        keyMatrix[j][k++] = i + 'A';  
  
        if (k == SIZE)  
        {  
            k = 0;  
            j++;  
        }  
    }  
}
```

```
void findPosition(char ch, int *row, int *col)
```

```
{  
    int i, j;  
  
    if (ch == 'J')  
        ch = 'I';  
  
    for (i = 0; i < SIZE; i++)  
    {  
        for (j = 0; j < SIZE; j++)
```

```

    {
        if (keyMatrix[i][j] == ch)
        {
            *row = i;
            *col = j;
            return;
        }
    }
}

```

```

void encrypt(char a, char b)

```

```

{
    int r1, c1, r2, c2;

    findPosition(a, &r1, &c1);
    findPosition(b, &r2, &c2);

    if (r1 == r2)
    {
        printf("%c%c", keyMatrix[r1][(c1 + 1) % 5], keyMatrix[r2][(c2 + 1) % 5]);
    }
    else if (c1 == c2)
    {

```



```
        printf("%c%c", keyMatrix[(r1 + 1) % 5][c1], keyMatrix[(r2 + 1) % 5][c2]);  
    }  
    else  
    {  
        printf("%c%c", keyMatrix[r1][c2], keyMatrix[r2][c1]);  
    }  
}
```

```
int main()  
{  
    char key[100], text[100];  
    int i;  
  
    printf("Enter Key: ");  
    scanf("%s", key);  
  
    printf("Enter Plain Text: ");  
    scanf("%s", text);  
  
    generateKeyMatrix(key);  
  
    printf("\nKey Matrix:\n");  
    for (i = 0; i < 5; i++)  
    {
```

```

    int j;

    for (j = 0; j < 5; j++)

        printf("%c ", keyMatrix[i][j]);

    printf("\n");
}

printf("\nCipher Text: ");

for (i = 0; text[i] != '\0'; i += 2)
{
    char a = toupper(text[i]);

    char b = (text[i + 1] != '\0') ? toupper(text[i + 1]) : 'X';

    if (a == 'J') a = 'I';

    if (b == 'J') b = 'I';

    if (a == b)

        encrypt(a, 'X');

    else

        encrypt(a, b);

}

return 0;
}

```

### **OUTPUT:**

```
Enter Key: BEST
Enter Plain Text: KESHAVARAM

Key Matrix:
B E S T A
C D F G H
I K L M N
O P Q R U
V W X Y Z

Cipher Text: PDAFBZTUTN

...Program finished with exit code 0
Press ENTER to exit console.█
```

### **RESULT:**

Thus the Playfair cipher substitution technique had been implemented successfully.

### **EX. NO: 3**

### **IMPLEMENTATION OF HILL CIPHER**

#### **AIM:**

To write a C program to implement the hill cipher substitution techniques.

#### **ALGORITHM:**

STEP-1: Read the plain text and key from the user.

STEP-2: Split the plain text into groups of length three.

STEP-3: Arrange the keyword in a 3\*3 matrix.

STEP-4: Multiply the two matrices to obtain the cipher text of length three.

STEP-5: Combine all these groups to get the complete cipher text.

#### **PROGRAM:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    int a[3][3] = {
```

```
        {6, 24, 1},
```

```
        {13, 16, 10},
```

```
        {20, 17, 15}
```

```
    };
```

```
    int b[3][3] = {
```

```
        {8, 5, 10},
```

```
        {21, 8, 21},
```

```
        {21, 12, 8}
```

```
    };
```

```
    int c[3], d[3];
```

```
char msg[20];

int i, j, t;

printf("Enter Plain Text (3 letters): ");

scanf("%s", msg);

if (strlen(msg) != 3)
{
    printf("Please enter exactly 3 uppercase letters.\n");
    return 0;
}

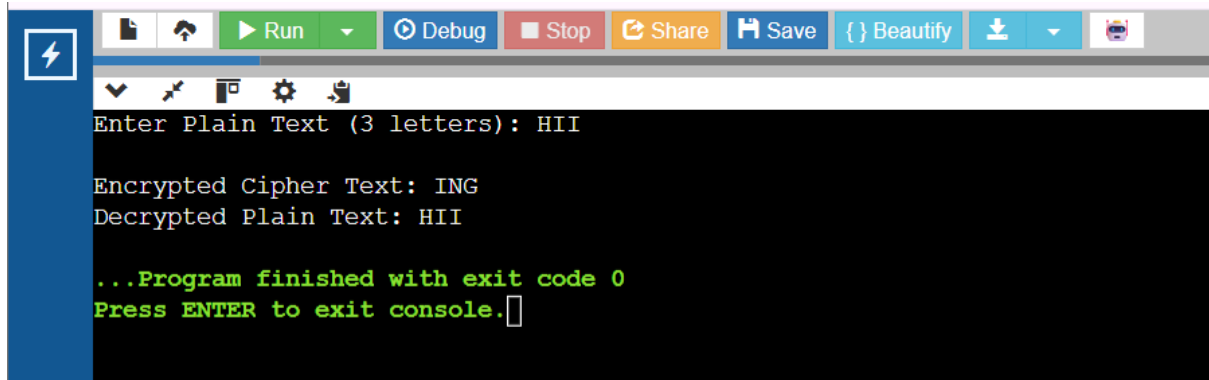
for (i = 0; i < 3; i++)
{
    if (msg[i] >= 'a' && msg[i] <= 'z')
        msg[i] = msg[i] - 32;

    c[i] = msg[i] - 'A';
}

for (i = 0; i < 3; i++)
{
    t = 0;
    for (j = 0; j < 3; j++)
    {
        t += a[i][j] * c[j];
    }
    d[i] = t % 26;
}
```

```
printf("\nEncrypted Cipher Text: ");  
for (i = 0; i < 3; i++)  
{  
    printf("%c", d[i] + 'A');  
}  
  
for (i = 0; i < 3; i++)  
{  
    t = 0;  
    for (j = 0; j < 3; j++)  
    {  
        t += b[i][j] * d[j];  
    }  
    c[i] = t % 26;  
}  
  
printf("\nDecrypted Plain Text: ");  
for (i = 0; i < 3; i++)  
{  
    printf("%c", c[i] + 'A');  
}  
  
return 0;  
}
```

**OUTPUT:**



The screenshot shows a code editor interface with a toolbar at the top containing buttons for Run, Debug, Stop, Share, Save, Beautify, and a download icon. Below the toolbar is a dark-themed console window displaying the following text:

```
Enter Plain Text (3 letters): HII
Encrypted Cipher Text: ING
Decrypted Plain Text: HII
...Program finished with exit code 0
Press ENTER to exit console.
```

## **RESULT:**

Thus the hill cipher substitution technique had been implemented successfully in C.

## **EX. NO: 4**

### **IMPLEMENTATION OF VIGENERE CIPHER**

#### **AIM:**

To implement the Vigenere Cipher substitution technique using C program.

#### **ALGORITHM:**

STEP-1: Arrange the alphabets in row and column of a 26\*26 matrix.

STEP-2: Circulate the alphabets in each row to position left such that the first letter is attached to last.

STEP-3: Repeat this process for all 26 rows and construct the final key matrix. STEP-4: The keyword and the plain text is read from the user.

STEP-5: The characters in the keyword are repeated sequentially so as to match with that of the plain text.

STEP-6: Pick the first letter of the plain text and that of the keyword as the row indices and column indices respectively.

STEP-7: The junction character where these two meet forms the cipher character. STEP-8: Repeat the above steps to generate the entire cipher text.

#### **PROGRAM:**

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
void encipher();
```

```
void decipher();
```

```
int main()
```

```
{
```

```
    int choice;
```



```
while (1)
{
    printf("\n1. Encrypt Text");
    printf("\n2. Decrypt Text");
    printf("\n3. Exit");

    printf("\n\nEnter Your Choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            encipher();
            break;

        case 2:
            decipher();
            break;

        case 3:
            exit(0);

        default:
            printf("\nInvalid Choice!\n");
    }
}

return 0;
}
```

```

void encipher()
{
    char input[50], key[50];
    int i, j;

    printf("\nEnter Plain Text: ");
    scanf("%s", input);

    printf("Enter Key: ");
    scanf("%s", key);

    printf("Encrypted Text: ");

    for (i = 0, j = 0; i < strlen(input); i++, j++)
    {
        if (j == strlen(key))
            j = 0;

        printf("%c", ((toupper(input[i]) - 'A' + toupper(key[j]) - 'A') % 26) + 'A');
    }

    printf("\n");
}

```

```

void decipher()
{
    char input[50], key[50];
    int i, j;

```

```
printf("\nEnter Cipher Text: ");
```

```
scanf("%s", input);
```

```
printf("Enter Key: ");
```

```
scanf("%s", key);
```

```
printf("Decrypted Text: ");
```

```
for (i = 0, j = 0; i < strlen(input); i++, j++)
```

```
{
```

```
    if (j == strlen(key))
```

```
        j = 0;
```

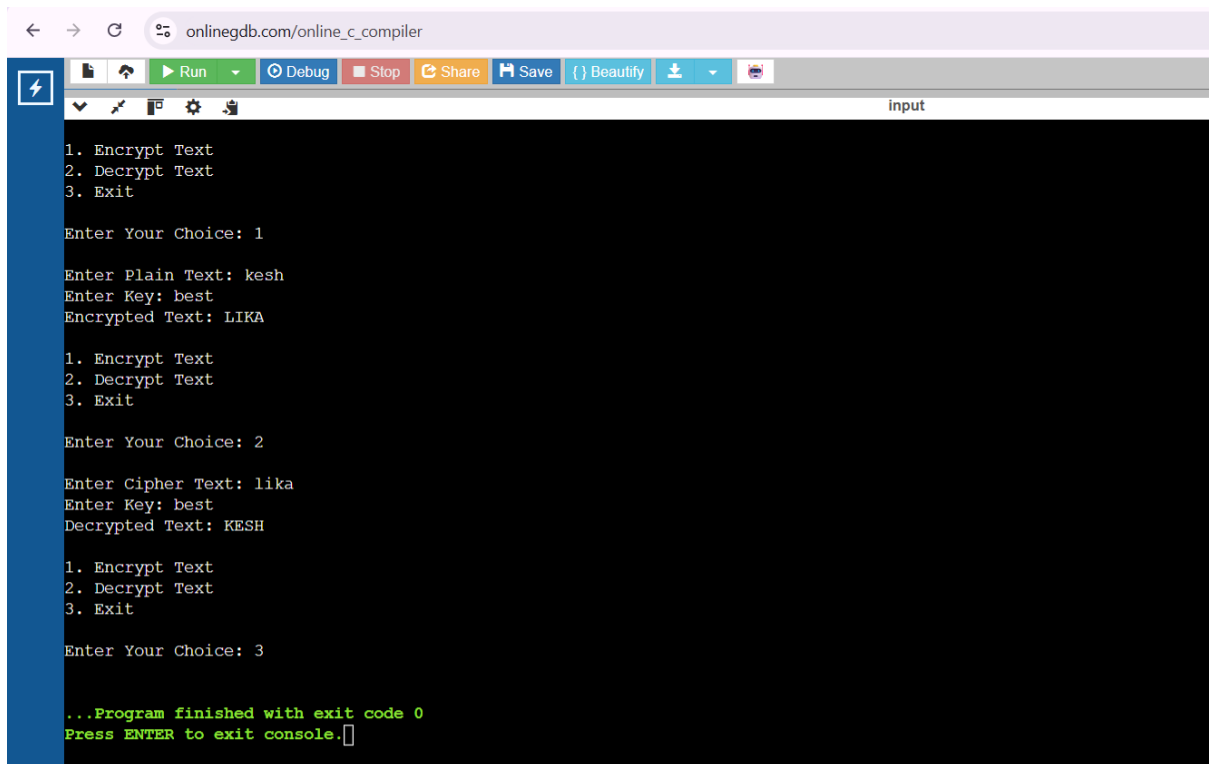
```
    printf("%c", ((toupper(input[i]) - 'A' - (toupper(key[j]) - 'A') + 26) % 26) + 'A');
```

```
}
```

```
printf("\n");
```

```
}
```

## OUTPUT:



The screenshot shows a web browser window with the address bar displaying "onlinegdb.com/online\_c\_compiler". The browser's toolbar includes buttons for "Run", "Debug", "Stop", "Share", "Save", "Beautify", and a download icon. Below the toolbar is a dark-themed code editor with a blue vertical margin on the left. The code in the editor is a C program for a Vigenere cipher. The program's output is displayed in the console area on the right, which has a title bar labeled "input". The output shows the program running through three iterations: 1. Encrypting "kesh" with key "best" to produce "LIKA". 2. Decrypting "lika" with key "best" to produce "KESH". 3. Exiting the program. The final output line is "...Program finished with exit code 0" and "Press ENTER to exit console.".

```
1. Encrypt Text
2. Decrypt Text
3. Exit

Enter Your Choice: 1

Enter Plain Text: kesh
Enter Key: best
Encrypted Text: LIKA

1. Encrypt Text
2. Decrypt Text
3. Exit

Enter Your Choice: 2

Enter Cipher Text: lika
Enter Key: best
Decrypted Text: KESH

1. Encrypt Text
2. Decrypt Text
3. Exit

Enter Your Choice: 3

...Program finished with exit code 0
Press ENTER to exit console.
```

## RESULT:

Thus the Vigenere Cipher substitution technique had been implemented successfully.

## **EX. NO: 5**

### **IMPLEMENTATION OF RAIL FENCE – ROW & COLUMN TRANSFORMATION TECHNIQUE**

#### **AIM:**

To write a C program to implement the rail fence transposition technique.

#### **ALGORITHM:**

STEP-1: Read the Plain text.

STEP-2: Arrange the plain text in row columnar matrix format.

STEP-3: Now read the keyword depending on the number of columns of the plain text.

STEP-4: Arrange the characters of the keyword in sorted order and the corresponding columns of the plain text.

STEP-5: Read the characters row wise or column wise in the former order to get the cipher text.

#### **PROGRAM:**

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    int i, j, k, l;
```

```
    char a[100], c[100], d[100];
```

```
    printf("\n\t\tRAIL FENCE CIPHER\n");
```

```
    printf("\nEnter the Plain Text: ");
```

```
    scanf("%s", a);
```

```
    l = strlen(a);
```

```

/* Encryption */

j = 0;

for (i = 0; i < l; i++)
{
    if (i % 2 == 0)
        c[j++] = a[i];
}

for (i = 0; i < l; i++)
{
    if (i % 2 != 0)
        c[j++] = a[i];
}

c[j] = '\0';

printf("\nEncrypted Text : %s\n", c);

/* Decryption */

if (l % 2 == 0)
    k = l / 2;
else
    k = (l / 2) + 1;

j = 0;
for (i = 0; i < k; i++)
{
    d[j] = c[i];
}

```

```
        j += 2;
    }

    j = 1;
    for (i = k; i < l; i++)
    {
        d[j] = c[i];
        j += 2;
    }

    d[l] = '\0';

    printf("Decrypted Text : %s\n", d);

    return 0;
}
```

## OUTPUT:



The screenshot shows a code editor interface with a dark background. The top toolbar includes buttons for Run, Debug, Stop, Share, Save, Beautify, and a download icon. Below the toolbar, the console output is displayed in a monospaced font. The text in the console reads: "RAIL FENCE CIPHER", "Enter the Plain Text: KESH", "Encrypted Text : KSEH", "Decrypted Text : KESH", "...Program finished with exit code 0", and "Press ENTER to exit console." followed by a cursor icon.

```
RAIL FENCE CIPHER

Enter the Plain Text: KESH

Encrypted Text : KSEH
Decrypted Text : KESH

...Program finished with exit code 0
Press ENTER to exit console.
```

## RESULT:

Thus the rail fence algorithm had been executed successfully



## **EX. NO: 6**

### **IMPLEMENTATION OF DES**

#### **AIM:**

To write a C program to implement Data Encryption Standard (DES) using C Language.

#### **ALGORITHM:**

STEP-1: Read the 64-bit plain text.

STEP-2: Split it into two 32-bit blocks and store it in two different arrays.

STEP-3: Perform XOR operation between these two arrays.

STEP-4: The output obtained is stored as the second 32-bit sequence and the original second 32-bit sequence forms the first part.

STEP-5: Thus the encrypted 64-bit cipher text is obtained in this way. Repeat the same process for the remaining plain text characters.

#### **PROGRAM:**

```
import java.util.Scanner;

import java.security.SecureRandom;

import javax.crypto.Cipher;

import javax.crypto.KeyGenerator;

import javax.crypto.SecretKey;

import javax.crypto.spec.SecretKeySpec;

public class Main {

    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Plain Text: ");

        String message = sc.nextLine();
```

```
KeyGenerator keyGen = KeyGenerator.getInstance("DES");
SecureRandom random = SecureRandom.getInstance("SHA1PRNG");
random.setSeed("12345".getBytes());

keyGen.init(56, random);
SecretKey secretKey = keyGen.generateKey();

byte[] raw = secretKey.getEncoded();

System.out.print("DES Key: ");
for (byte b : raw)
    System.out.printf("%02X", b);
System.out.println();

Cipher cipher = Cipher.getInstance("DES");

cipher.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(raw, "DES"));
byte[] encrypted = cipher.doFinal(message.getBytes());

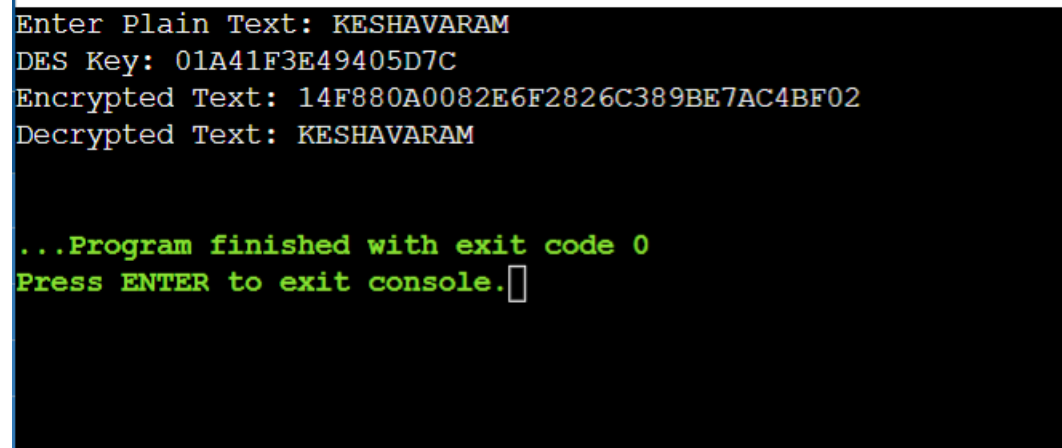
System.out.print("Encrypted Text: ");
for (byte b : encrypted)
    System.out.printf("%02X", b);
System.out.println();

cipher.init(Cipher.DECRYPT_MODE, new SecretKeySpec(raw, "DES"));
byte[] decrypted = cipher.doFinal(encrypted);

System.out.println("Decrypted Text: " + new String(decrypted));
```

```
        sc.close();  
    }  
}
```

### **OUTPUT:**



```
Enter Plain Text: KESHAVARAM  
DES Key: 01A41F3E49405D7C  
Encrypted Text: 14F880A0082E6F2826C389BE7AC4BF02  
Decrypted Text: KESHAVARAM  
  
...Program finished with exit code 0  
Press ENTER to exit console.
```

### **RESULT:**

Thus the data encryption standard algorithm had been implemented successfully using C language.

## **EX. NO: 7**

### **IMPLEMENTATION OF RSA**

#### **AIM:**

To write a C program to implement the RSA encryption algorithm.

#### **ALGORITHM:**

STEP-1: Select two co-prime numbers as p and q.

STEP-2: Compute n as the product of p and q.

STEP-3: Compute  $(p-1)*(q-1)$  and store it in z.

STEP-4: Select a random prime number e that is less than that of z.

STEP-5: Compute the private key, d as  $e * \text{mod-1}(z)$ .

STEP-6: The cipher text is computed as  $\text{message} * \text{mod } n$ . STEP-7: Decryption is done as  $\text{cipherdmod } n$ .

#### **PROGRAM:**

```
#include <stdio.h>
```

```
long long modExp(long long base, long long exp, long long mod)
```

```
{
```

```
    long long result = 1;
```

```
    base = base % mod;
```

```
    while (exp > 0)
```

```
    {
```

```
        if (exp % 2 == 1)
```

```
            result = (result * base) % mod;
```

```
        exp = exp / 2;
```

```
        base = (base * base) % mod;
```

```
    }
```

```
    return result;
}
```

```
int gcd(int a, int b)
{
    while (b != 0)
    {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}
```

```
int modInverse(int e, int phi)
{
    int d;

    for (d = 1; d < phi; d++)
    {
        if ((e * d) % phi == 1)
            return d;
    }

    return -1;
}
```

```
int isPrime(int n)
{

```

```
if (n < 2)
    return 0;
```

```
for (int i = 2; i * i <= n; i++)
{
    if (n % i == 0)
        return 0;
}
```

```
return 1;
}
```

```
int main()
```

```
{
    int p, q;
```

```
    printf("Enter first prime number: ");
    scanf("%d", &p);
```

```
    printf("Enter second prime number: ");
    scanf("%d", &q);
```

```
    if (!isPrime(p) || !isPrime(q) || p == q)
    {
        printf("Invalid prime numbers.\n");
        return 0;
    }
```

```
    int n = p * q;
```

```
int phi = (p - 1) * (q - 1);
```

```
int e;
```

```
for (e = 2; e < phi; e++)
```

```
{
```

```
    if (gcd(e, phi) == 1)
```

```
        break;
```

```
}
```

```
int d = modInverse(e, phi);
```

```
printf("\nPublic Key = (%d, %d)\n", e, n);
```

```
printf("Private Key = (%d, %d)\n", d, n);
```

```
char msg[100];
```

```
printf("\nEnter message (lowercase): ");
```

```
scanf("%s", msg);
```

```
long long cipher[100];
```

```
printf("\nEncrypted Message:\n");
```

```
int i;
```

```
for (i = 0; msg[i] != '\0'; i++)
```

```
{
```

```
    int m = msg[i];
```

```
    cipher[i] = modExp(m, e, n);
    printf("%lld ", cipher[i]);
}

printf("\n\nDecrypted Message:\n");

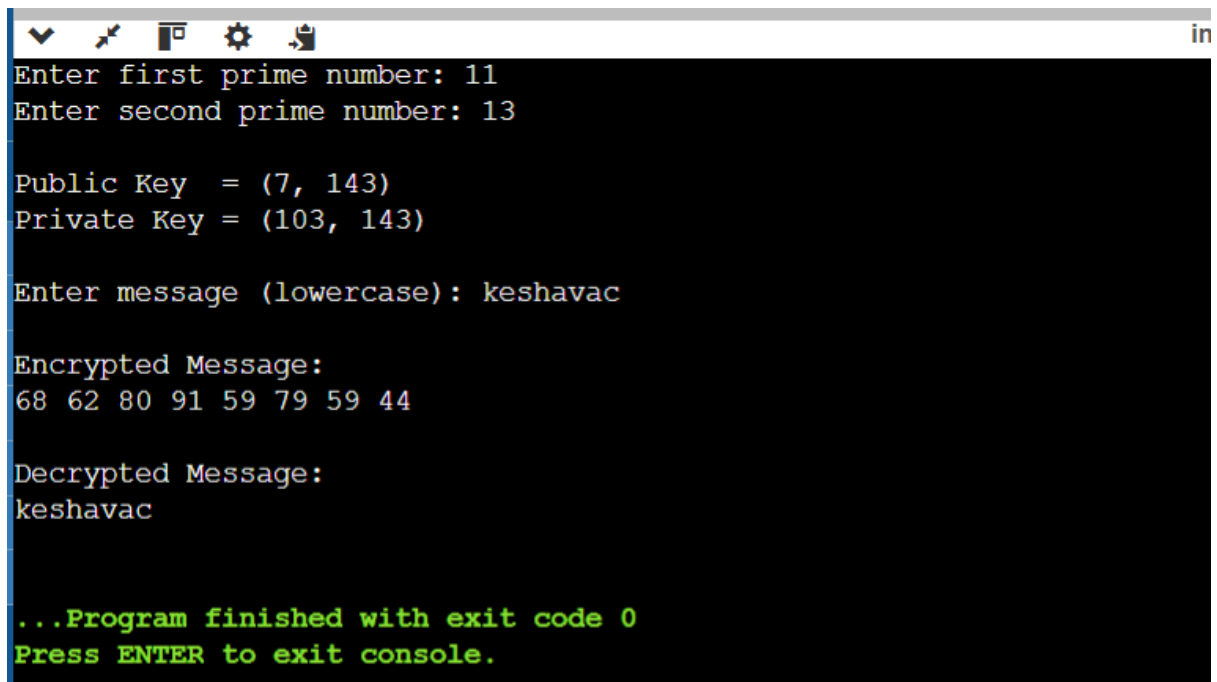
for (i = 0; msg[i] != '\0'; i++)
{
    char ch = (char)modExp(cipher[i], d, n);
    printf("%c", ch);
}

printf("\n");

return 0;
}
```



## OUTPUT:

A screenshot of a terminal window with a dark background and light-colored text. The window has a title bar with standard icons (minimize, maximize, close) and a small 'in' icon on the right. The text inside the terminal shows the execution of an RSA encryption program. It starts with prompts for two prime numbers, 11 and 13. It then displays the calculated Public Key (7, 143) and Private Key (103, 143). Next, it prompts for a message, 'keshavac', and shows the encrypted message as a sequence of numbers: 68 62 80 91 59 79 59 44. The decrypted message is shown as 'keshavac'. The program ends with a green message: '...Program finished with exit code 0' and 'Press ENTER to exit console.'

```
Enter first prime number: 11
Enter second prime number: 13

Public Key = (7, 143)
Private Key = (103, 143)

Enter message (lowercase): keshavac

Encrypted Message:
68 62 80 91 59 79 59 44

Decrypted Message:
keshavac

...Program finished with exit code 0
Press ENTER to exit console.
```

## RESULT:

Thus the C program to implement RSA encryption technique had been implemented successfully

